

Fuzzy-Service

Diseño Técnico (Propuesta)

1) Nombre y propósito

Nombre: `fuzzy-service`

Propósito: Consumir lecturas **consolidadas y validadas** del `sensor-service` (vía **MQTT**), aplicar **lógica difusa** sobre variables configurables (MFs y reglas), y **emitir instrucciones** al `actuator-service` vía **HTTP endpoint**. El `actuator-service` publicará dichas instrucciones en el **broker MQTT** para que las consuma el firmware **ESP32**.

2) Glosario

- **Lógica difusa (Fuzzy):** control por grados de pertenencia (0–1), no binario.
- **MF (Membership Function):** función que define una etiqueta lingüística (p.ej., "frío", "alto").
- **Defuzzificación:** conversión de un conjunto difuso en un valor numérico (p.ej., **centroid**).
- **MQTT:** protocolo pub/sub ligero para IoT.
- **ACK:** acuse de recibo/ejecución exitosa.
- **JWT/Scopes:** token y permisos granulares (p.ej., `fuzzy.read` , `fuzzy.write` , `fuzzy.execute`).

3) Alcance y relaciones

- **Sensor-Service** → **MQTT (topic consolidado)** → **Fuzzy-Service (worker suscrito)**
- **Fuzzy-Service** → **HTTP** → **Actuator-Service** → **MQTT (publica instrucciones)** → **ESP32**
- **Auth-Service:** emite JWT; el `fuzzy-service` **valida localmente** y aplica **scopes**.

ACKs por responsabilidad

- **Sensor-Service:** ACKs de ingestión/publicación de telemetría.
- **Actuator-Service:** ACKs de publicación a MQTT y ACKs de los ESP32.

- **Fuzzy-Service:** ACK **interno** del envío exitoso de la instrucción al Actuator (HTTP 2xx).

4) Clean Architecture (capas, responsabilidades y estructura)

[Web/API]	→	[Application]	→	[Domain]	→	[Infraestructura]
FastAPI/JWT		UseCases/DTOs		Entidades/VOs		Mongo/MQTT/HTTP
Validación scopes		Orquestación		Motor de reglas		Repos + adaptadores
Rutas CRUD/simulación		Políticas/validaciones		MFs/defuzz/AST reglas		HTTP Client → Actuator

4.1 Capa Web/API (FastAPI)

- **Qué expone:**
 - CRUD de **variables, términos (MFs), reglas, rutinas, mapeos de actuadores**.
 - **Simulación** (`/evaluate` y `/simulate`).
 - **Import/Export** de sistemas.
- **Seguridad:**
 - **Autenticación:** validación **local** de **JWT** con llave pública del `auth-service` .
 - **Autorización (granular por scopes):**
 - Lecturas: `fuzzy.read`
 - Escrituras: `fuzzy.write`
 - Ejecución/evaluación: `fuzzy.execute`
 - Admin “completa” scopes faltantes.

4.2 Capa Application

- **Use cases** (ejemplos):

`CreateVariable` , `CreateTerm` , `CreateRule` , `CreateRoutine` , `MapActuator` ,
`PublishSystem` , `EvaluateSnapshot` , `SimulateSequence` , `SendInstructionToActuator` .
- **Responsabilidad:** orquestar flujos, aplicar validaciones de aplicación, invocar repos y adaptadores.

4.3 Capa Domain

- **Entidades/VOs puros:**

`FuzzySystem` , `Variable (input|output)` , `Term` , `Rule (antecedent/consequent)` ,

`Routine` , `ActuatorMapping` , `Instruction` .

- **Motor Fuzzy (puro):**
 - Soporta MFs: `triangle` , `trapezoid` , `gauss` , `gbell` , `sigmoid` , `piecewise/custom` .
 - Operadores: `AND/OR/NOT` con `min/max` o `prod/prob_or` (configurable por sistema).
 - Defuzz: **centroid** (por defecto), `bisector` , `mom` (mean of maxima), `som` , `lom` .
- **Sin dependencias de frameworks** (test unitarios sencillos).

4.4 Capa Infrastructure

- **Repos Mongo** (`Motor` async o `PyMongo` sync).
- **MQTT adapter** para **suscribirse** (solo **entrada** desde `sensor-service`).
Recomendado: `asyncio-mqtt` (alternativa: `paho-mqtt`).
- **HTTP client** → **actuator-service** para **enviar instrucciones**.
- **Serializadores** import/export.

5) Librerías

- **API:** FastAPI, `python-jose` o `PyJWT` (JWT + JWKS fetch).
- **Fuzzy:** `scikit-fuzzy (skfuzzy)` + NumPy.
- **MQTT:** `asyncio-mqtt` (recomendado) / `paho-mqtt` (alternativa).
- **Mongo:** `motor` (async) / `pymongo` (sync).
- **Validación:** Pydantic v2 (modelos/DTOs).
- **Contenedores:** Docker + Docker Compose (.env).
- **(Futuro)** Prometheus/Grafana, K8s Secrets/Vault.

6) Modelos de datos (MongoDB) — JSON de ejemplo

Nota: se modela por colección para permitir reuso, versionado y filtros.

Campos `*At` en ISO8601. `systemId` para agrupar.

6.1 `fuzzy_systems`

```
{
  "_id": "sys_veg_2025_v1",
  "name": "Vegetativo Día",
  "version": 1,
  "status": "published",
  "operators": { "and": "min", "or": "max", "not": "complement" },
  "defuzzMethod": "centroid",
```

```

    "createdBy": "user123",
    "createdAt": "2025-08-12T10:00:00Z",
    "updatedAt": "2025-08-12T15:00:00Z"
  }

```

6.2 **variables** (inputs/outputs)

```

{
  "_id": "var_temp_air",
  "systemId": "sys_veg_2025_v1",
  "name": "temp_air",
  "type": "input",
  "range": [15, 40],
  "units": "°C",
  "termIds": ["term_temp_low", "term_temp_ok", "term_temp_high"]
}

```

6.3 **terms** (MFs)

```

{
  "_id": "term_temp_high",
  "label": "high",
  "mf": {
    "type": "trapezoid",
    "params": [28, 30, 40, 40] // a,b,c,d
  }
}

```

MFs soportadas y **params**:

- **triangle** : [a, b, c]
- **trapezoid** : [a, b, c, d]
- **gauss** : [mean, sigma]
- **gbell** : [a, b, c]
- **sigmoid** : [a, c] (pendiente y centro)
- **piecewise/custom** : [{x, y}, ...] (ordenados)

6.4 **rules**

```

{
  "_id": "rule_cool_air",
  "systemId": "sys_veg_2025_v1",
  "weight": 1.0,
  "antecedent": {
    "op": "AND",

```

```

"args": [
  { "var": "temp_air", "is": "high" },
  { "var": "hum", "is": "ok_or_high", "op": "OR", "alts": [
    { "var": "hum", "is": "ok" },
    { "var": "hum", "is": "high" }
  ]
},
],
"consequent": [
  { "output": "fan_power", "is": "high" },
  { "output": "led_on", "is": "off" }
],
"description": "Si temp alta y humedad ok/alta → subir ventilador, apagar LED"
}

```

El antecedente puede representarse como AST (op/args) o como una mini-DSL; aquí se muestra AST JSON.

6.5 routines

```

{
  "_id": "routine_cooldown",
  "systemId": "sys_veg_2025_v1",
  "name": "cooldown",
  "steps": [
    { "stepOrder": 1, "output": "fan_power", "mapTo": "set_power", "durationMs": 300000 },
    { "stepOrder": 2, "output": "pump_on", "mapTo": "on", "durationMs": 60000 }
  ]
}

```

6.6 actuator_mappings

```

{
  "_id": "map_fan_1",
  "systemId": "sys_veg_2025_v1",
  "output": "fan_power",
  "actuatorId": "fan_1",
  "esp32Id": "esp32_zone_1",
  "mappingType": "percentage",
  "params": { "minPercent": 0, "maxPercent": 100, "threshold_on": 10 }
}

```

Tipos:

- `boolean` (on/off con umbrales)
- `percentage` (0–100)
- `pwm_string` (si el firmware lo pide)
- `position` / `setpoint` (p.ej., % válvula)

6.7 `audit_logs` (evaluaciones)

```
{
  "_id": "aud_001",
  "systemId": "sys_veg_2025_v1",
  "evalId": "eval-uuid-123",
  "inputs": { "temp_air": 30.5, "hum": 58, "lux": 8000 },
  "memberships": { "temp_air": { "high": 0.82, "ok": 0.18 } },
  "firingStrengths": { "rule_cool_air": 0.82 },
  "defuzzOutputs": { "fan_power": 73.4, "led_on": 0.0 },
  "actionsProduced": [
    {
      "actuatorId": "fan_1",
      "action": "set_power",
      "durationMs": 300000,
      "metadata": { "power_percent": 73.4 }
    }
  ],
  "timestamp": "2025-08-12T15:34:20Z",
  "sendToActuatorHttp": { "status": 202, "requestId": "req-xyz" }
}
```

ACKs de MQTT/ESP32 quedan en el actuator-service. El fuzzy-service solo registra el resultado del POST al actuator.

7) Contratos de integración

7.1 MQTT (solo **entrada** para fuzzy-service)

- **Topic telemetría consolidada** (publicado por `sensor-service`):
`hydro/{esp32Id}/telemetry/aggregate`

```
{
  "esp32Id": "esp32_zone_1",
  "timestamp": "2025-08-12T12:00:00Z",
  "sensors": {
    "ph": 6.8,
    "ec": 1.25,
    "temp_air": 28.5,
```

```
"hum": 60.2,  
"lux": 12000,  
"water_level_cm": 10,  
"temp_water": 22.3  
}  
}
```

7.2 HTTP — Fuzzy → Actuator

- **Endpoint (ejemplo sugerido por actuator-service):**

POST /api/actuators/commands

Body (instrucción):

```
{  
  "commandId": "uuid-1234",  
  "esp32Id": "esp32_zone_1",  
  "actuatorId": "fan_1",  
  "action": "set_power",  
  "durationMs": 300000,  
  "metadata": {  
    "source": "fuzzy-service:v1",  
    "power_percent": 73.4,  
    "fuzzyRule": "rule_cool_air",  
    "evalId": "eval-uuid-123"  
  },  
  "timestamp": "2025-08-12T15:34:20Z"  
}
```

- **Respuesta Actuator (sugerida):**

```
{ "accepted": true, "requestId": "req-xyz-789" }
```

8) Flujo de operación (runtime)

1. **Worker MQTT** (TLS) del `fuzzy-service` se suscribe a `hydro/+/telemetry/aggregate`.
2. Llega mensaje → **normaliza** rangos/unidades y **clamp** al rango de la variable.
3. Carga (cache in-process) el **sistema “published”** desde Mongo (`fuzzy_systems` + relaciones).
4. **Inferencia difusa** (skfuzzy): MFs → reglas → agregación → **defuzz**.
5. **Mapeo a acciones** con `actuator_mappings` y **rutina** (si aplica) → **instrucciones**.
6. **HTTP POST** al `actuator-service` con cada instrucción.
7. Guarda **auditoría** (inputs, membresías, reglas activas, outputs, acciones, estado POST).

8. (Opcional) el `fuzzy-service` puede **consultar** estado en `actuador-service` si se requiere.
-

9) Endpoints del `fuzzy-service` (con scopes)

| Prefijo sugerido: `/api/fuzzy`

9.1 Sistemas

- `POST /systems` (*fuzzy.write*) — crear sistema (draft).
- `GET /systems` (*fuzzy.read*) — listar (filtros: name, status, version).
- `GET /systems/{id}` (*fuzzy.read*) — detalle.
- `PUT /systems/{id}` (*fuzzy.write*) — actualizar (solo draft).
- `POST /systems/{id}/publish` (*fuzzy.write*) — publicar versión.
- `POST /systems/{id}/export` (*fuzzy.read*) — export JSON paquete.
- `POST /systems/import` (*fuzzy.write*) — importar paquete.

9.2 Variables / Términos (MFs)

- `POST /variables` (*fuzzy.write*)
- `GET /variables` (*fuzzy.read*)
- `GET /variables/{id}` (*fuzzy.read*)
- `PUT /variables/{id}` (*fuzzy.write*)
- `DELETE /variables/{id}` (*fuzzy.write*)
- `POST /terms` (*fuzzy.write*)
- `GET /terms` (*fuzzy.read*)
- `GET /terms/{id}` (*fuzzy.read*)
- `PUT /terms/{id}` (*fuzzy.write*)
- `DELETE /terms/{id}` (*fuzzy.write*)

9.3 Reglas / Rutinas / Mapeos

- `POST /rules` (*fuzzy.write*)
- `GET /rules` (*fuzzy.read*)
- `GET /rules/{id}` (*fuzzy.read*)
- `PUT /rules/{id}` (*fuzzy.write*)
- `DELETE /rules/{id}` (*fuzzy.write*)
- `POST /routines` (*fuzzy.write*)
- `GET /routines` (*fuzzy.read*)
- `GET /routines/{id}` (*fuzzy.read*)

- `PUT /routines/{id}` (*fuzzy.write*)
- `DELETE /routines/{id}` (*fuzzy.write*)
- `POST /actuator-mappings` (*fuzzy.write*)
- `GET /actuator-mappings` (*fuzzy.read*)
- `GET /actuator-mappings/{id}` (*fuzzy.read*)
- `PUT /actuator-mappings/{id}` (*fuzzy.write*)
- `DELETE /actuator-mappings/{id}` (*fuzzy.write*)

9.4 Evaluación / Simulación

- `POST /systems/{id}/evaluate` (*fuzzy.execute*)

Request:

```
{ "inputs": { "temp_air": 30.2, "hum": 58, "lux": 8000 }, "routine": "cooldo
wn" }
```

Response:

```
{
  "evalId": "uuid",
  "memberships": { "temp_air": { "high": 0.82, "ok": 0.18 } },
  "firing": { "rule_cool_air": 0.82 },
  "outputs": { "fan_power": 73.4, "led_on": 0.0 },
  "actions": [
    { "actuatorId": "fan_1", "action": "set_power", "durationMs": 300000,
      "metadata": { "power_percent": 73.4 } }
  ]
}
```

- `POST /systems/{id}/simulate` (*fuzzy.execute*)

Request:

```
{ "sequence": [ { "t": 0, "inputs": { "temp_air": 28 } }, { "t": 60, "inputs": {
"temp_air": 31 } } ] }
```

9.5 Auditoría

- `GET /audits` (*fuzzy.read*) — filtros: `systemId`, `from`, `to`, `ruleId`, `actuatorId`, `sentStatus`.

10) Seguridad

- **Autenticación:** JWT validado **localmente** con **JWKS** del `auth-service`.
- **Autorización:** por **scopes** (`fuzzy.read`, `fuzzy.write`, `fuzzy.execute`); **admin** supe scopes.

- **MQTT (worker): TLS** con usuario/clave o mTLS (broker).
 - **Secrets: .env + Docker Compose** por ahora. (*Futuro: K8s Secrets o Vault*).
-

11) Rendimiento

- **Precálculo** de funciones de membresía y vectorización (**NumPy**).
 - **Cache in-process** con TTL de la configuración publicada (sin Redis).
 - **Rate limiting por actuador** (en memoria) para evitar flapping.
 - **Idempotencia:** `commandId` en cada instrucción.
-

12) Testing

- **Unit tests:** MF/defuzz/reglas/AST.
 - **Integration:** worker MQTT local + Mongo + mock de actuator-service (HTTP).
 - **Contract tests:** payload de `/api/actuators/commands`.
 - **E2E (opcional):** secuencia simulada → audit.
-

13) Configuración (.env — ejemplo)

```
MONGO_URI=mongodb://mongo:27017
MONGO_DB=fuzzy
MQTT_HOST=broker
MQTT_PORT=8883
MQTT_TLS_ENABLED=true
MQTT_USERNAME=fuzzy
MQTT_PASSWORD=secret
SENSOR_TOPIC=hydro/+/telemetry/aggregate
ACTUATOR_BASEURL=http://actuator-service:8080
JWT_JWKS_URL=http://auth-service/.well-known/jwks.json
JWT_AUDIENCE=fuzzy-service
JWT_ISSUER=https://auth.example
```

14) Desarrollo y despliegue

- **Desarrollo:** Docker Compose (Mongo, Broker MQTT, Fuzzy, Actuator mock).
 - **Producción inicial:** Contenedores Docker.
 - **Futuro:** Kubernetes + K8s Secrets o Vault + Prometheus/Grafana.
-

15) Funcionalidades futuras

- **Observabilidad (Prometheus/Grafana):** métricas `evals_total`, `latency`, `commands_sent`, etc.

- **Gestión de secretos** con K8s Secrets o Vault.
 - **Trazas (OpenTelemetry)** si crece la complejidad.
 - **Auto-tuning / neuro-fuzzy**.
 - **Multi-worker** con particionado por `esp32ld`.
-

16) Notas de implementación Fuzzy (skfuzzy)

- MFs via `skfuzzy.membership`: `trimf`, `trapmf`, `gaussmf`, `gbellmf`, `sigmf`.
 - Reglas: combinar pertenencias con `np.minimum/maximum` o operadores probabilísticos.
 - Agregación de salidas por término → **defuzz** con `skfuzzy.defuzz` (`centroid`, `bisector`, `mom`, etc.).
 - **Dominio de salida**: definir universo (p.ej., `np.linspace(0, 100, 1001)` para %).
 - Validar **coherencia**: rangos, cobertura de MFs, solapamientos y huecos.
-

Si quieres, en el siguiente paso te genero un **esqueleto de proyecto** (FastAPI + worker MQTT + repos Mongo + cliente HTTP al actuador) con los **modelos Pydantic** y routers iniciales para que partan de ahí.